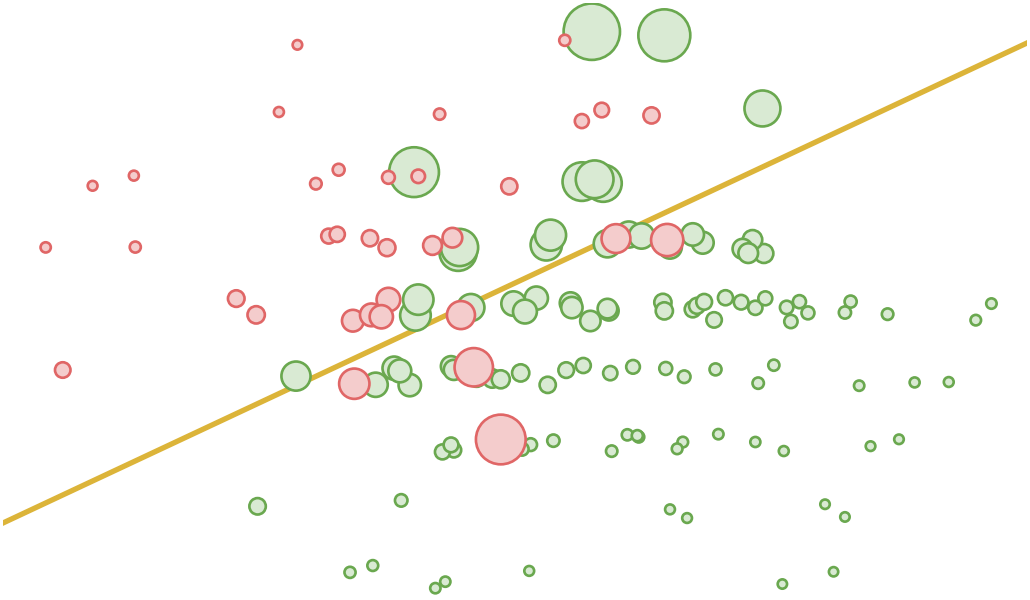
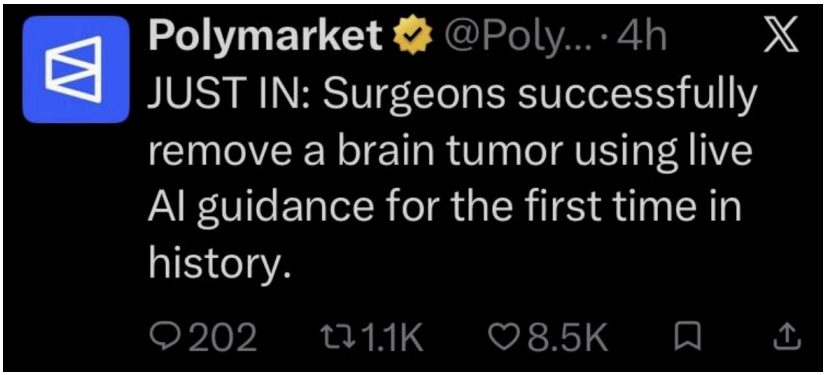


UNIT 01 · CHAPTER 2

Loss Functions

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

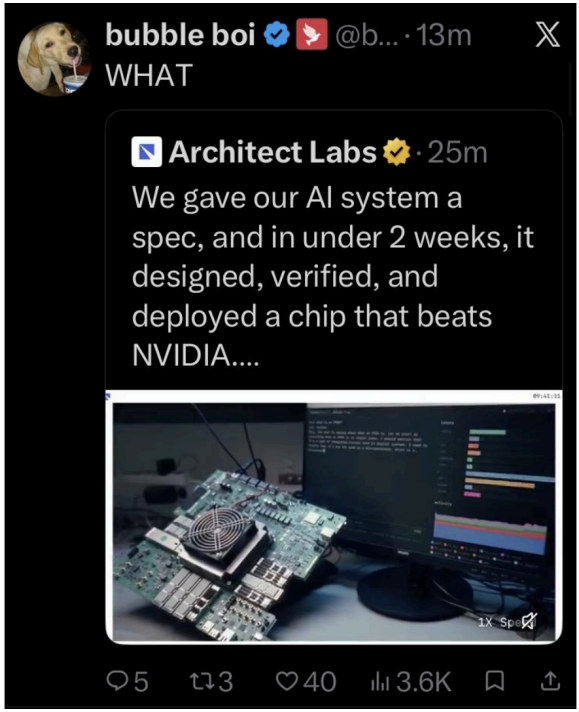
AI in the News

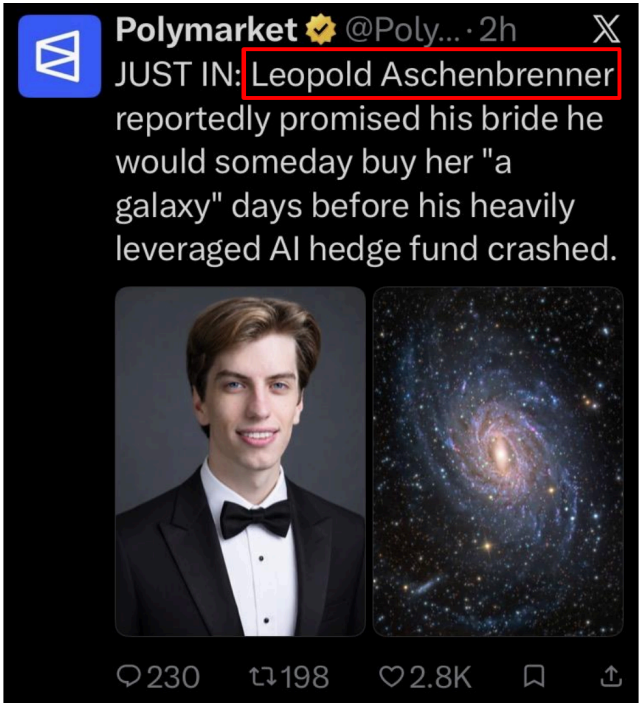




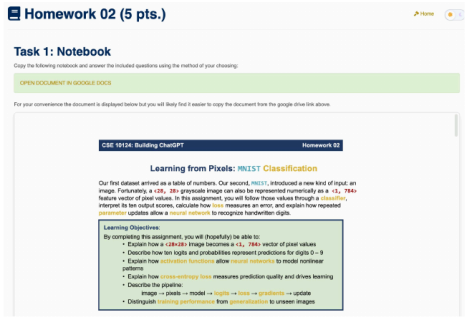


AI in the News





Logistics: Homework02, Lab01



Homework 02 (5 pts.)

Home

Task 1: Notebook

Copy the following notebook and answer the included questions using the method of your choosing:

OPEN DOCUMENT IN GOOGLE DOCS

For your convenience the document is displayed below but you will likely find it easier to copy the document from the google drive link above.

CSE 10124: Building ChatGPT — Homework 02

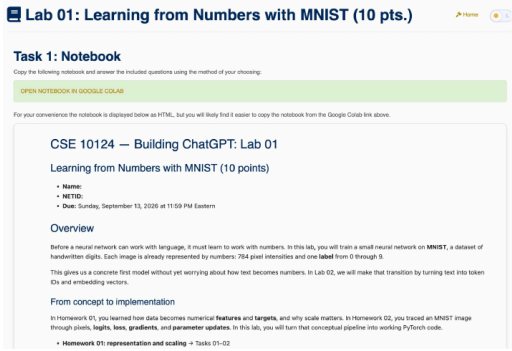
Learning from Pixels: MNIST Classification

Our first dataset arrived as a table of numbers. Our second, MNIST, introduced a new kind of input: an image. Fortunately, a 28, 28× grayscale image can also be represented numerically as a 1, 784× feature vector of pixel values. In this assignment, you will follow those values through a classifier, interpret its ten output scores, calculate how loss measures an error, and explain how repeated parameter updates allow a neural network to recognize handwritten digits.

Learning Objectives:

By completing this assignment, you will (hopefully) be able to:

- Explain how a 28×28× image becomes a 1, 784× vector of pixel values
- Describe how ten logits and probabilities represent predictions for digits 0 – 9
- Explain how activation functions allow neural networks to model nonlinear patterns
- Explain how cross-entropy loss measures prediction quality and drives learning
- Describe the pipeline:
  image → pixels → model → logits → loss → gradients → update
- Distinguish training performance from generalization to unseen images



Lab 01: Learning from Numbers with MNIST (10 pts.)

Home

Task 1: Notebook

Copy the following notebook and answer the included questions using the method of your choosing:

OPEN NOTEBOOK IN GOOGLE COLAB

For your convenience the notebook is displayed below as HTML, but you will likely find it easier to copy the notebook from the Google Colab link above.

CSE 10124 — Building ChatGPT: Lab 01

Learning from Numbers with MNIST (10 points)

- **Name:**
- **NETID:**
- **Due:** Sunday, September 13, 2026 at 11:59 PM Eastern

Overview

Before a neural network can work with language, it must learn to work with numbers. In this lab, you will train a small neural network on **MNIST**, a dataset of handwritten digits. Each image is already represented by numbers: 784 pixel intensities and one **label** from 0 through 9.

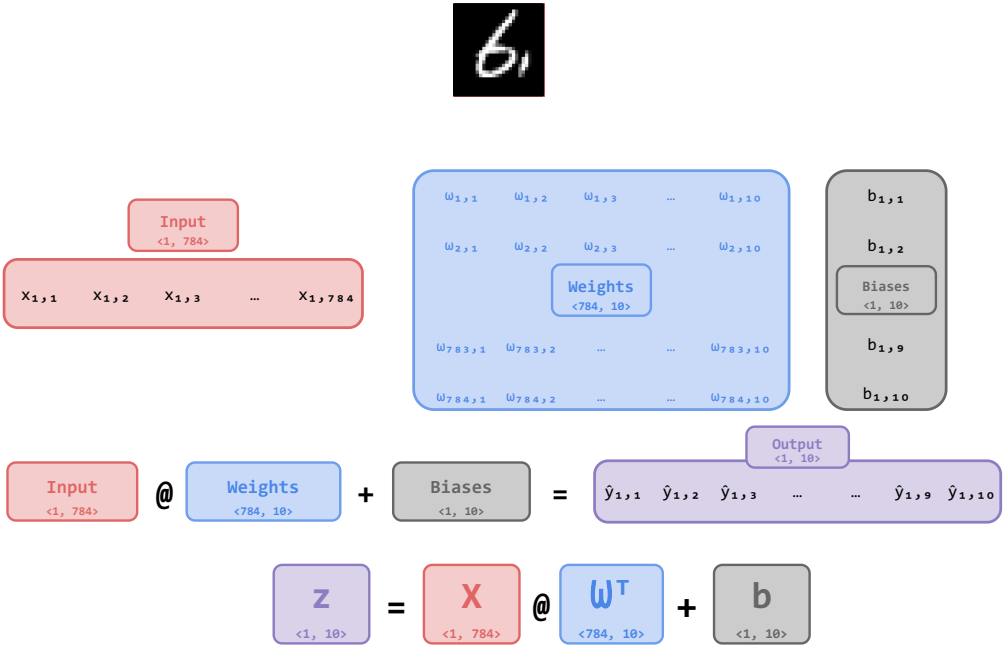
This gives us a concrete first model without yet worrying about how text becomes numbers. In Lab 02, we will make that transition by turning text into token IDs and embedding vectors.

From concept to implementation

In Homework 01, you learned how data becomes numerical **features** and **targets**, and why scale matters. In Homework 02, you traced an MNIST image through pixels, **logits**, **loss**, **gradients**, and **parameter updates**. In this lab, you will turn that conceptual pipeline into working PyTorch code.

- **Homework 01: representation and scaling** → Tasks 01–02

Programming Day 01: Architecture

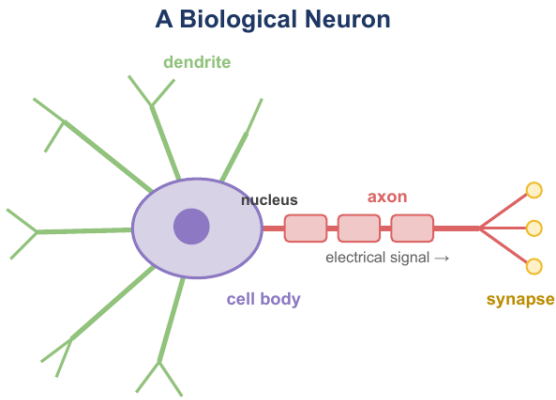


Our model has a single **layer**, going from a `<1, 784>` **input vector** down to a `<1, 10>` **output vector**. The equation for a single layer is the bottom row:
$z = X @ W^T + b$.

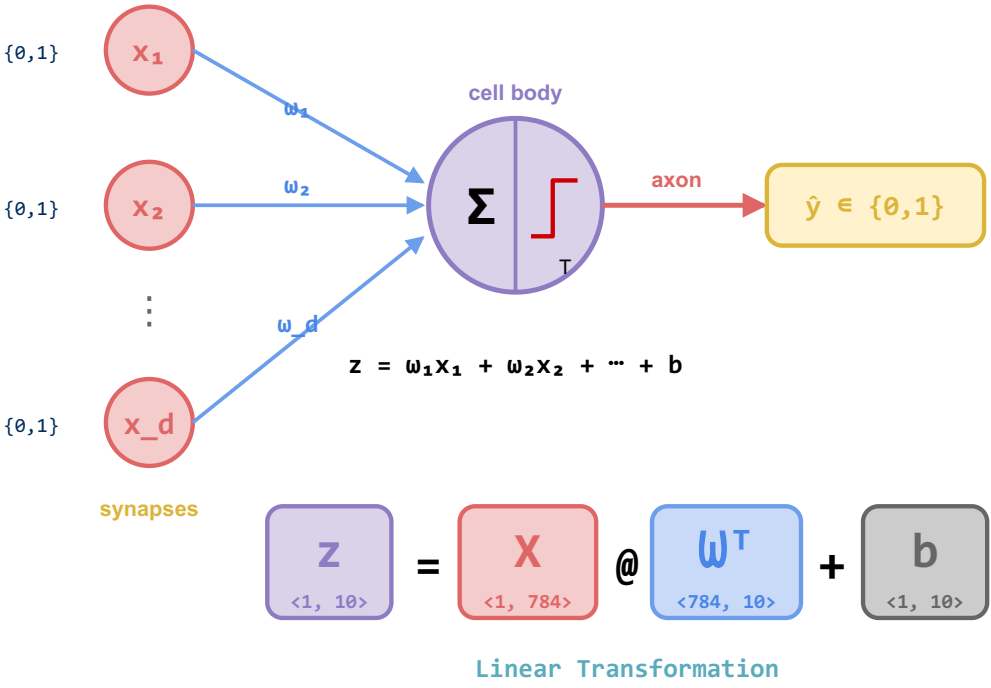
McCulloch–Pitts Artificial Neuron (1943)

What a **neural network** is trying to model is the way a human brain works, by combining and chaining together **artificial neurons** in **layers**.

- **Layer:** a group of **neurons** that process inputs in parallel and pass their outputs to another **layer**.
- **Neuron:** a computational unit with a set of **parameters** (**weights** and **biases**) that creates a weighted sum of the input.

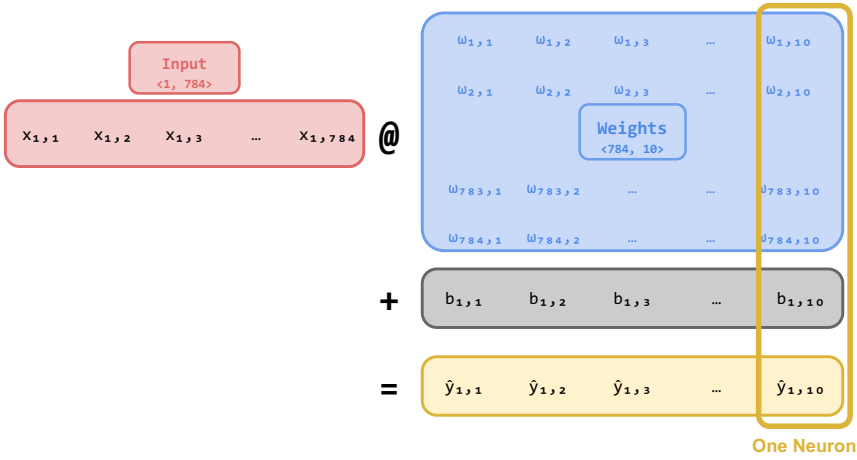


A McCulloch–Pitts Neuron



The weighted sum inside the cell body is the same **linear transformation** as our layer, one **neuron** at a time.

Artificial Neurons

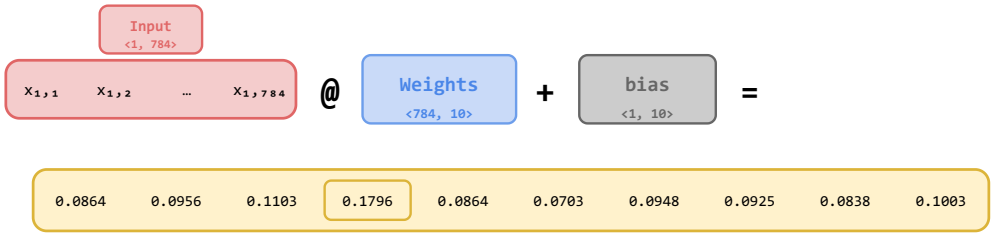


Our model could be said to have **1** **layer** with **10** **neurons**, with each **neuron** producing one of the output numbers.

Producing an Output Prediction

We have `10` possible outputs **(targets/classes)** that we can produce, one for each number.







Representing an Output Prediction and the Ground Truth



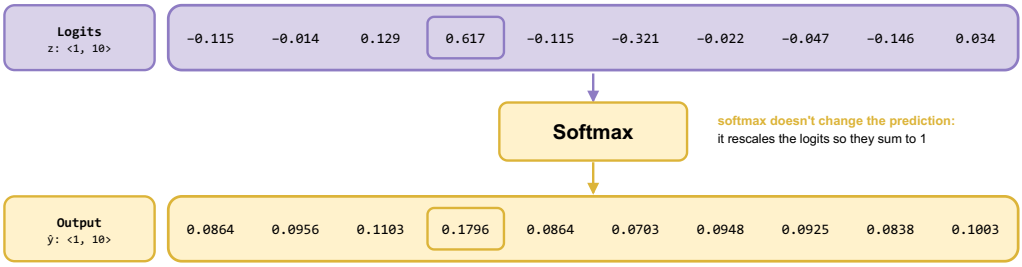
Output ŷ: <1, 10>	0.0864	0.0956	0.1103	**0.1796**	0.0864	0.0703	0.0948	0.0925	0.0838	0.1003
	0	1	2	3	4	5	6	7	8	9
Ground Truth y: <1, 10>	0.0	0.0	0.0	0.0	0.0	**1.0**	0.0	0.0	0.0	0.0

We return the **largest index value** as our prediction (`3` in this case), which we refer to as $\hat{y}$.

Our **ground truth vector** is a **one-hot vector** with the **same size** as the **output vector**, with a `1` in the correct **index** and `0`s everywhere else. We refer to this vector as **y**.

Softmax and Logits

On Friday we saw an additional function called **Softmax**, whose purpose was to transform our `<1, 10>` **output vector** into a **probability distribution**. Before we apply the **softmax** function, our output is called **logits**.



- **Logits:** the model's raw, unnormalized outputs, with one dimension for each possible **class**.
- **Softmax:** a function that converts **logits** into nonnegative probabilities that sum to `1`.

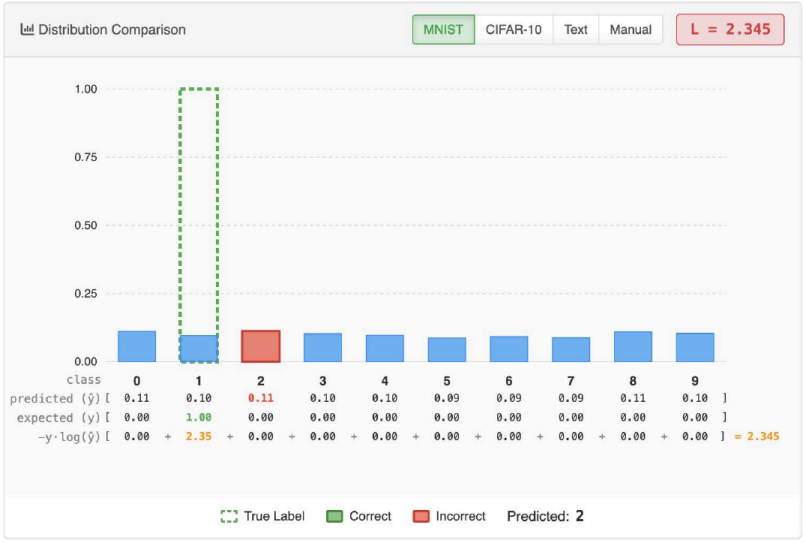
Loss Functions

- **Loss:** a single numerical measure of how well a model's prediction matches the expected answer.
- **Loss Function:** a function that calculates our **Loss**. **Cross-Entropy** is the **loss function** we'll use primarily in this course.

$$\mathcal{L}_{ce} = -\sum_{i=1}^{10} y_i \cdot \log(\hat{y}_i)$$

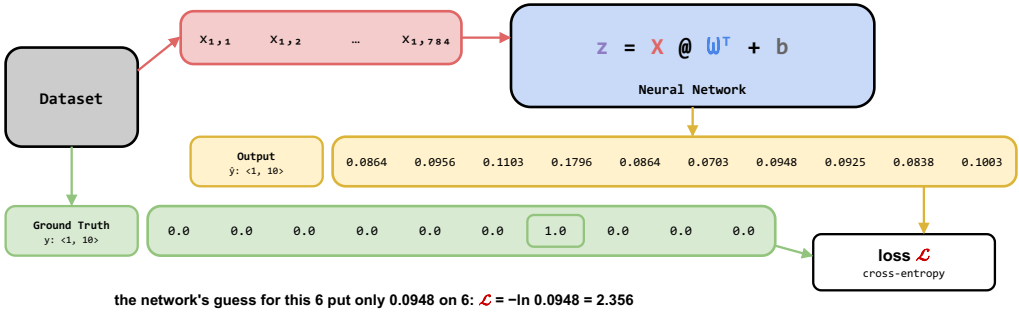
Cross-Entropy: the loss function used to train LLMs is called "Cross-Entropy Loss" and it's one of the most common loss functions. Heavily based on the principle of "entropy" as used in Information Theory, it's a measure of how surprising our answer is relative to our ground truth, based on the entropy formula $-\Sigma\ p(x)\ \log_2\ p(x)$. Mostly outside the scope of the class, but it's really not *that* complicated.

Loss Function: Cross Entropy



Cross Entropy Visualizer

Mathematical Formulation of a Neural Network



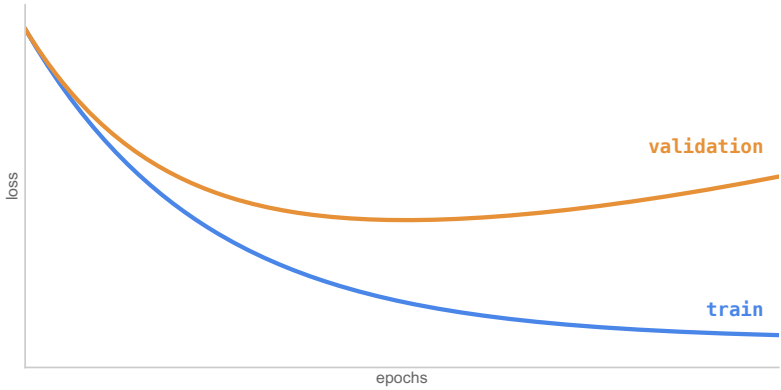
$$2.356 = \mathcal{L}_{ce} = -\sum_{i=1}^{10} y_i \cdot \log(\hat{y}_i)$$

We can write our **loss** as a function of our **input** and **parameters**:

$$2.356 = \mathcal{L}_{ce} = -\sum_{i=1}^{10} y_i \cdot \log(\text{softmax}(XW^T + b)_i)$$

Formulating a **neural network** mathematically in this manner is **critical to actually training one**. **Wednesday** we'll dig more into this, and it will be the "mathiest" lecture in this course.

Training a Neural Network



Over the course of many **epochs** (one training "step" of a neural network), we'd expect to see our **loss steadily decrease**, which is indicative of our **predictions getting better**.

Training a Model

Training a model lets it practice over and over on the same set of data, ideally **improving its performance** over time! It does this by updating its **parameters**.

